



NLP : Natural Language Processing



Before all :

let's introduce ourselves

- Me :
 - Training
 - Work experience
- You :
 - Initial training
 - Work/internship experience
 - Level of knowledge in Machine learning, Programming, AI, NLP
 - Expectations for this course



Outline

1. Introduction
 - What is NLP?
 - Use Cases : real-word applications
2. Text Preprocessing
3. Text representation
 - Classical approaches
 - Embeddings
4. Core NLP tasks
 - Sequence labeling
 - Text classification/clustering
5. Deep learning & LLMs for NLP
 - Neural network architectures & LLMs
6. Capstone Project



Objective 1 : Master foundations

- Introduction/definitions
- Use cases
- Text processing



Objective 2 : Exploiting text data for applications

- Text representation
- Core NLP tasks



Objective 3 : Understand more advanced models

- Deep learning models for NLP
- LLMs



Objective 4 : Implement real-world solutions

- Capstone project



Evaluation methods





Evaluations

- A series of labs + assignments to be completed after lessons no. 2; 4; 6
 - All your work should be available in on github repository
- Participation grade
- Capstone project : last day
- Exam (?!)



My rules

- => Practice-oriented class : “practice makes perfect”
- => Avoid Chatter : talk about the course, ask questions!
- => No wrong question/answer
- => Give your thoughts, share your knowledge
- => **DO NOT use AI if not mentioned** => try by yourself first : penalties for those who use it without indication



Outline

1. Introduction
 - What is NLP?
 - Use Cases : real-word applications
2. Text Preprocessing
3. Text representation
 - Classical approaches
 - Embeddings
4. Core NLP tasks
 - Sequence labeling
 - Text classification/clustering
5. Deep learning & LLMs for NLP
 - Neural network architectures & LLMs
6. Capstone Project



Part 1 : Introduction

Definitions & use cases

Understanding natural language processing

What is NLP

- **Natural Language Processing (NLP):** field of **Artificial Intelligence** that focuses on enabling computers to understand, analyze, generate, and interact with human language.
- **NLP challenges:**
 - Human language is rich, ambiguous, and context-dependent
 - Computers deal with numbers. But unlike numbers or images, words do not have a fixed meaning.





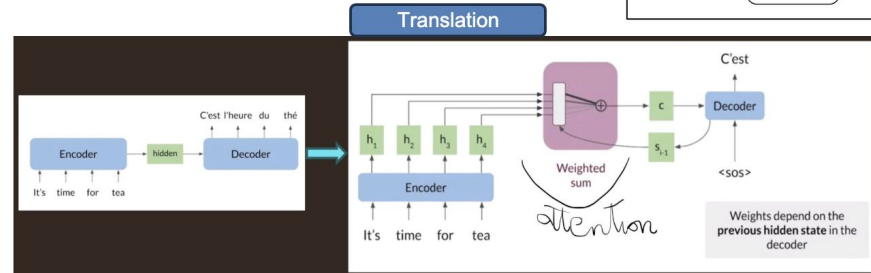
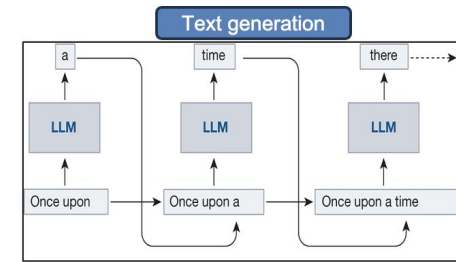
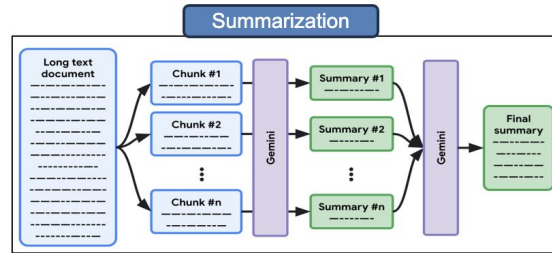
NLP vs Computer vision vs Tabular Machine Learning

- **Tabular ML** works with structured data -> **Numbers**
- **Computer vision** works with images -> **Pixels**
- **NLP** works with language -> **Text data**

Type	Data	Examples	Typical Models
NLP	Text, speech	Reviews, emails, chat	Transformers, RNNs
Computer Vision	Images, videos	Faces, objects, X-rays	CNNs, Vision Transformers
Tabular ML	Tables, numbers	Sales, prices, customer data	XGBoost, Random Forest

NLP use cases

- Text classification:
 - Reviews, questions...
- Chatbots:
 - <https://bot.tgvinoui.sncf/>
- Search engines:
 - Browsers
- Recommendation systems:
 - <https://www.amazon.fr/>



Prerequisites & environment setup for NLP

- **Prerequisites :**
 - Basics on ML 
 - Python programming 
- **Materials :**
 - Python + Notebook lab 
 - VS Code  or Anaconda  or Google colab  (<https://colab.research.google.com/>)
 - Install libraries `nlTK`, `spaCy`, `transformers`, `huggingface`
 - Github account 
 - Hugging face account + create access token => <https://huggingface.co/> 
 - Mistral : create account + access token => <https://console.mistral.ai/> 
- **TODO :** Follow instructions in <https://github.com/ababacaryoro/nlp-course/blob/main/SETUP.md>



Part 2 : Text preprocessing

Techniques & fundamentals

Manipulate textual data



Handling text data

- **Structure of text data :**
 - Strings + characters => Words / tokens => Sentences => Documents
- **Why do we preprocess text?**
 - Raw text is noisy, inconsistent, and hard for machines to interpret.
 - Preprocessing transforms raw language into clean, structured, usable data
- **Basic text operations:**

Operation	Example
Lowercasing	"Apple" → "apple"
Trimming spaces	" hello " → "hello"
Replacing text	"don't" → "do not"
Removing punctuation	"price!" → "price"
Splitting text	"I like NLP" → ["I", "like", "NLP"]
Removing numbers	"model v2" → "model"
Removing URLs	"visit http://..." → "visit"



Handling text data

- **Application in Python** : suppose "s" is a string variable
 - **len(s)**: Returns the number of characters in the string. Includes letters, spaces, and symbols
 - **s.upper()**: Converts all letters to uppercase
 - **s.lower()**: Converts all letters to lowercase
 - **s.split()**: Splits the string into a list of words (by spaces by default)
 - **s.strip()**: Removes leading and trailing spaces
 - **s.find(substring)**: Returns the position (index) of a substring
 - **s.replace(old, new)**: Replaces all occurrences of a word or character
 - **s.startswith(prefix)**: Checks if the string starts with a given prefix
 - **s.endswith(suffix)**: Checks if the string ends with a given suffix
 - **s.isupper()**: Checks if all letters are uppercase
 - **s.isalpha()**: Checks if the string contains only letters (A-Z)
 - **s.isdigit()**: Checks if the string contains only numbers



Regular expressions

- **Regular Expression (Regex)** : special sequence of characters that helps you match or find patterns in a text
- They are used for **searching, manipulating, or editing** text based on patterns.
- **Use cases:**
 - Remove URLs from reviews
 - Extract order numbers or specific informations
 - Clean product codes



Regular expressions

- Most common regular expressions (meta-characters):

Pattern	Meaning	Example
.	Matches any single character (except newline)	"c.t" matches "cat", "cut", "c9t"
^	Matches the start of the string	"^Hello" matches "Hello world" but not "Say Hello"
\$	Matches the end of the string	"world\$" matches "Hello world"
*	0 or more repetitions	"go*" matches "g", "go", "goo", "gooo"
+	1 or more repetitions	"go+" matches "go", "goo", "gooo" but not "g"
?	0 or 1 repetition	"colou?r" matches "color" and "colour"
[]	Matches any one of the characters inside	"[aeiou]" matches any vowel
\d	Matches any digit (0–9)	\d\d matches "23" in "Order 23"
\w	Matches letters, numbers, or _	\w+ matches "Product_123"
\s	Matches any whitespace	"Hello\sWorld" matches "Hello World"
\b	Word boundary (start or end of a word)	\bcat\b matches "cat" but not "catalog"

See more on REGEX : <https://docs.python.org/3/library/re.html>



Regular expressions

- **Regex in Python:**
 - The `re` module allows you to search, extract, replace, and validate text using regular expressions.

Function	Purpose	Example	Result
<code>re.search()</code>	Finds the first occurrence of a pattern in a string	<code>re.search(r"\d+", "Order 456")</code>	Matches "456"
<code>re.findall()</code>	Returns all matches as a list	<code>re.findall(r"\d+", "Orders 12 and 34")</code>	<code>['12', '34']</code>
<code>re.sub()</code>	Replaces matched text	<code>re.sub(r"\w+@\w+\.\w+", "[EMAIL]", "Contact me at test@gmail.com")</code>	"Contact me at [EMAIL]"
<code>re.split()</code>	Splits text using a pattern	<code>re.split(r"[;,]", "apple, orange; banana")</code>	<code>['apple', ' orange', ' banana']</code>
<code>re.match()</code>	Checks if a pattern matches at the start of a string	<code>re.match(r"\d+", "123abc")</code>	Match
<code>re.match()</code>	No match if pattern is not at the beginning	<code>re.match(r"\d+", "abc123")</code>	No match



LAB1: Text data manipulation & Regex

- **Materials :**
 - https://github.com/ababacaryoro/nlp-course/blob/main/lab1/lab1_text_manipulation.ipynb
- **Instructions :**
 - Run the notebook (local with uv, Codespaces or Colab, see SETUP.md).
 - Commit it with its outputs in your repository nlp-course-LASTNAME and push.
 - Then notify the teacher by e-mail (repository link in your first e-mail only).



Tokenization

- **Definition :** Process of splitting a text or a sentence in smaller chunks/units called tokens.
- **Tokens** can be:
 - Words
 - Subwords
 - Characters
 - Sentences
- **Example:**
 - "I love natural language processing" → ["I", "love", "natural", "language", "processing"]
- **Why tokenizing ?**
 - Machine learning models cannot process raw text. They work with tokens.



Tokenization

- **Types of Tokenization:**
 - **Word Tokenization:** "This product is amazing" → ["This", "product", "is", "amazing"]
 - **Sentence Tokenization:** "I like it. It works well." → ["I like it.", "It works well."]
 - **Subword Tokenization:** "unbelievable" → ["un", "believe", "able"]
 - **Character Tokenization:** "cat" → ["c", "a", "t"]
- **Tokenization Challenges:**
 - **Contractions:** "don't" → "do" + "not" or "don't"
 - **Multi-word expression:** "New York" → one token or two?
 - **Emojis, hashtags, URLs:** 😊 | #Metoo | <https://www.datacamp.com/>
- **Modern models (BERT, GPT) use subword tokenization:**
 - Byte Pair Encoding (BPE)
 - WordPiece
 - SentencePiece



Stemming/Lemmatization

- These are **techniques** that aim to **reduce words** to their **base form** so that different variations of the same word are treated as one.
- **Examples:**
 - connect, connected, connecting → connect
 - run, runs, running → run
- **Main goal :**
 - Reduce vocabulary size
 - Improve model generalization
 - Capture the true meaning of words



Stemming/Lemmatization

- **Stemming:**

- Removes the end of a word to get its root form.
- It uses simple rules, not dictionaries.
- Examples:
 - playing → play
 - studies → studi
 - better → better
 - happiness → happi
- Characteristics
 - Fast
 - Simple
 - **Can produce non-real words**
- Different type of stemmers giving different outputs: Porter stemmer, Lancaster stemmer,...



Stemming/Lemmatization

- **Lemmatization:**

- Converts a word to its dictionary base form (lemma).
- It uses: grammar, vocabulary, part-of-speech.
- Examples:
 - running → run
 - better → good
 - was → be
- Characteristics
 - Slower
 - Accurate
 - **Produces real, meaningful words.**
- Depending on parameters of the lemmatizer, the output can be different:
 - “Running” give “run” if we look for a **verb** as root or “**running**” if we look for a **noun**



Stemming/Lemmatization

- **When to use which:**
 - Use **Stemming** when:
 - Speed is important
 - Small mistakes are acceptable
 - You are doing simple text classification → we just need to reduce the dimensions of data
 - Use **Lemmatization** when:
 - Meaning/interpretability is important
 - You need high-quality language understanding
 - You are using modern NLP models



Stop-words & special characters

- **Raw text** contains many elements that **do not carry useful meaning** for NLP models.
- **Cleaning** helps remove **noise** and **focus** on **what matters**.
- **Stopwords:**
 - Very **common words** that usually **do not add much meaning**.
 - **Examples:** *the, is, a, an, and, of, to, in, on, for*
 - **Stopwords removal** : “This is a very good product” → “very good product”
 - **Removing stopwords:**
 - Reduces vocabulary size
 - Improves focus on important words
 - Speeds up training
 - When NOT to Remove Stopwords:
 - Sentiment analysis: “not good” → removing “not” breaks the meaning
 - Question answering, language generation...



Stop-words & special characters

- **Special characters:**
 - **Include:**
 - Punctuation (!, . ?)
 - Symbols (\$ % @ #)
 - Emojis 😊
 - HTML tags (
)
 - URLs and mentions (http://, @user)
 - **Special characters removal :** "Great!!! 🥰🥰" → "great"
 - **Removing special characters :**
 - Standardizes text
 - Reduces sparsity
 - Improves model learning

Typical text data cleaning steps

- **Common cleaning pipeline:** raw text → "This product is AMAZING!!! 🥰 Visit <http://shop.com> now!!!"
 - Lowercase →
 - "this product is amazing!!! 🥰 visit <http://shop.com> now!!!"
 - Remove URLs →
 - "this product is amazing!!! 🥰 visit now!!!"
 - Remove punctuation & symbols & emojis →
 - "this product is amazing visit now"
 - Tokenize →
 - ["this", "product", "is", "amazing", "visit", "now"]
 - Remove stopwords →
 - ["product", "amazing", "visit"]
 - Stemming or lemmatization →
 - ["product", "amazing", "visit"]



LAB2

- **Materials :**

- https://github.com/ababacaryoro/nlp-course/blob/main/lab2/lab2_preprocessing_pipeline.ipynb

- **Instructions :**

- Run the notebook (local with uv, Codespaces or Colab, see SETUP.md).
- Commit it with its outputs in your repository nlp-course-LASTNAME and push.
- Then notify the teacher by e-mail (repository link in your first e-mail only).



Assignment 1

- Instructions :
 - Complete the 2 parts of the assignment :
 - https://github.com/ababacaryoro/nlp-course/blob/main/assignments/1/assignment1_part1_regex.ipynb
 - https://github.com/ababacaryoro/nlp-course/blob/main/assignments/1/assignment1_part2_nltk.ipynb
 - Make sure that your code can be ran without error
 - Send a mail to notify that the work is done **and push** into github



Part 3 : Text representation

From text to numbers

Convert text into machine-readable
numerical features

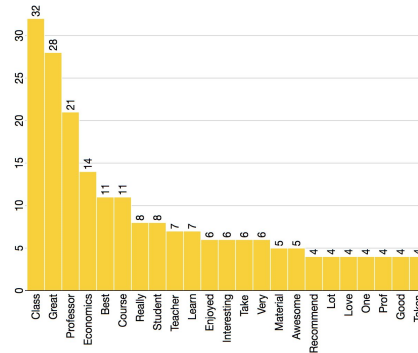


What is text data representation?

- **ML models** cannot work with **raw text** → only understand **numbers**.
- NLP classical **pipeline** : Text → Tokens → Numbers → Vectors → Models
- **Text analysis** options:
 - Graphical data visualizations
 - Bag of words
 - TF-IDF
 - Word/Sentence embedding

Text data visualization

- Bar diagram:
 - We count how often each word appears.
 - More efficient if we clean data first



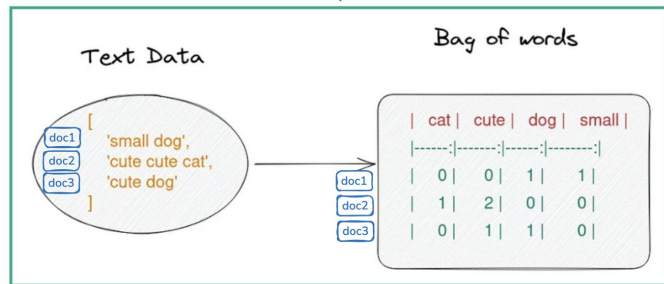
Text data visualization

- Before building models → visualize text statistics.
- Options :
 - Wordcloud:
 - Plot in which words are displayed with size proportional to frequency.
 - Gives a quick overview of the text's content



Bag of Words (BoW)

- **Bag of Words** represents a document by counting words.
- 3 steps:
 - **Tokenize** the documents
 - **Create vocabulary** : list of all unique tokens found in all documents
 - **Count the occurrence of all tokens** in every document recorded in a table, called Document-Term Matrix (DTM).





Bag of Words (BoW)

- **Properties:**
 - Ignores word order
 - **Only** counts frequency
 - Simple but powerful
- **Precautions:**
 - Clean the text data before, to reduce the dimension of the DTM & its sparsity
- **Limitations:**
 - **Sparsity:** with long documents, the DTM can be huge and contain multiple 0 for many documents
 - **Context:** the meaning of a word can depend on context and the other words surrounding it



TF-IDF

- TF-IDF stands for "Term Frequency-Inverse Document Frequency"
- Transform text into numeric vectors by:
 - Assigning a **weight** to each **word** in a **document**, based on its **frequency in the document** (TF) and its **frequency across all documents** (IDF).

$$TF_IDF(w, d) = TF(w, d) * \log\left(\frac{N}{DF(w)}\right)$$

where:

- **w**: a word in the document **d**
- **TF(w,d)**: the frequency of the word **w** in the document **d**
- **N**: the total number of documents in the corpus
- **DF(w)**: the number of documents that contain the word **w**



TF-IDF

$$TF_IDF(w, d) = TF(w, d) * \log\left(\frac{N}{DF(w)}\right)$$

where:

- **w**: a word in the document **d**
 - **TF(w,d)**: the frequency of the word **w** in the document **d**
 - **N**: the total number of documents in the corpus
 - **DF(w)**: the number of documents that contain the word **w**
- **TF-IDF:**
 - **Increases** weight for **rare** but **informative words**
 - **Decreases** weight for **common words** like “the”, “is”, “and”
 - **Improves** BoW by making **features** more **meaningful**.

N-grams

- N-grams capture word sequences in a corpus.
- Examples:
 - **Uni-grams:** "good", "product"
 - **Bi-grams:** "good product"
 - **Tri-grams:** "very good product"

N-grams identification

Corpus: I am happy because I am learning

Unigrams: { I, am, happy, because, learning }

Bigrams: { I am, am happy, happy because ... } ❌ I happy

Trigrams: { I am happy, am happy because, ... }

Sequence notation

Corpus: This is great ... teacher drinks tea.
 $w_1 \ w_2 \ w_3 \quad w_{498} \ w_{499} \ w_{500}$

$$w_1^m = w_1 \ w_2 \ \dots \ w_m$$

$$w_1^3 = w_1 \ w_2 \ w_3$$

$$w_{m-2}^m = w_{m-2} \ w_{m-1} \ w_m$$

N-grams

- N-grams models:

Unigram probability

Corpus: I am happy because I am learning

Size of corpus $m = 7$

$$P(I) = \frac{2}{7}$$

$$P(happy) = \frac{1}{7}$$

Probability of unigram:
$$P(w) = \frac{C(w)}{m}$$

Bigram probability

Corpus: I am happy because I am learning

$$P(am|I) = \frac{C(I\ am)}{C(I)} = \frac{2}{2} = 1$$

$$P(happy|I) = \frac{C(I\ happy)}{C(I)} = \frac{0}{2} = 0$$

$$P(learning|am) = \frac{C(am\ learning)}{C(am)} = \frac{1}{2}$$

Probability of a bigram:
$$P(y|x) = \frac{C(x\ y)}{\sum_w C(x\ w)} = \frac{C(x\ y)}{C(x)}$$



N-grams

- N-grams models:

Trigram Probability

Corpus: I am happy because I am learning

$$P(\text{happy} | I \text{ am}) = \frac{C(I \text{ am happy})}{C(I \text{ am})} = \frac{1}{2}$$

Probability of a trigram: $P(w_3 | w_1^2) = \frac{C(w_1^2 w_3)}{C(w_1^2)}$

$$C(w_1^2 w_3) = C(w_1 w_2 w_3) = C(w_1^3)$$

N-gram probability

Probability of N-gram: $P(w_N | w_1^{N-1}) = \frac{C(w_1^{N-1} w_N)}{C(w_1^{N-1})}$

$$C(w_1^{N-1} w_N) = C(w_1^N)$$



LAB3-1

- **Materials :**
 - https://github.com/ababacaryoro/nlp-course/blob/main/lab3/lab3_1_text_visualization_classical.ipynb
- **Instructions :**
 - Run the notebook (local with uv, Codespaces or Colab, see SETUP.md).
 - Commit it with its outputs in your repository nlp-course-LASTNAME and push.
 - Then notify the teacher by e-mail (repository link in your first e-mail only).



Embeddings

- **Bag of Words** and **TF-IDF** do not understand meaning.
- **Embeddings** represent words as dense vectors where:
 - Similar words have similar vectors.
- Generally, **embeddings** are created by **pre-trained models** built using **deep neural networks**.
- Embeddings:
 - Capture **similarity & relationships**
 - Improve model performance: **understand** language, not just **count words**
- Popular embeddings:
 - **Word2Vec**
 - **GloVe**
 - **BERT**



Embeddings

- **Word2Vec:**
 - **Neural network-based** model that learns word embeddings by predicting words from context.
 - **Two main models:**
 - **CBOW** – predicts a word from its context
 - **Skip-gram** – predicts context from a word
 - **Example:** Suppose we have the sentence → "The cat sat on the mat"

Using a window of size 2 for the context, the model learns:

- For "cat" → the context is: "The", "sat"
- For "sat" → context is: "cat", "on"

Embeddings

- **Word2Vec:**
 - **CBOW** architecture: predicts a word from its context

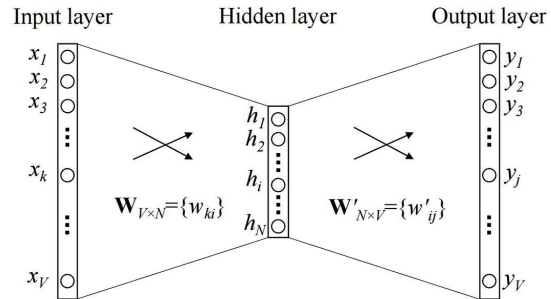
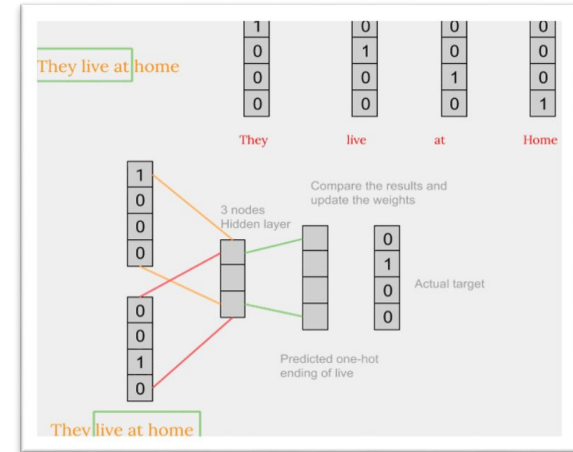
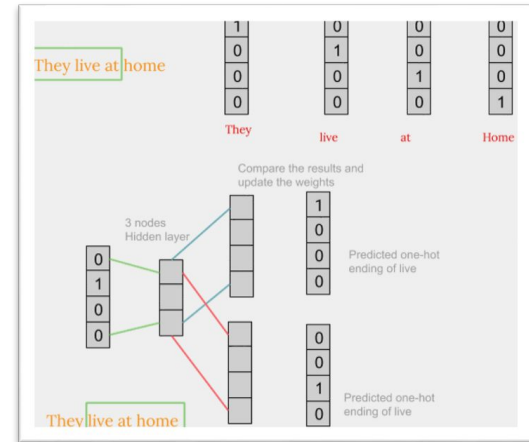
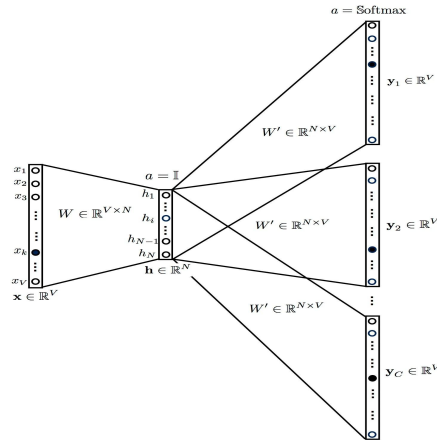


Figure 1: A simple CBOW model with only one word in the context



Embeddings

- **Word2Vec:**
 - **Skip-gram architecture:** predicts context from a word



Embeddings

- GloVe (Global Vectors):**
 - Learns embeddings from global **word co-occurrence statistics**.
 - Intuition:** words that occur frequently in similar contexts should have similar vector representations.

The fast cat wears no hat

The cat in the hat ran fast

	cat	fast	hat	in	no	ran	the	wears
cat	0							
fast	2	0						
hat	2	2	0					
in	1	1	1	0				
no	1	1	1	0	0			
ran	1	1	1	1	0	0		
the	3	3	3	2	1	2	1	
wears	1	1	1	1	1	0	1	0



	cat	fast			wears
cat	0				
fast	2				
.	.	.	.	X _{ij}	.
.	.	.	.	...	.
.	.	.	.		.
the					
wears					

X_{ij} tabulate the number of times word j occurs in the context of word i .

$X_i = \sum_k X_{ik}$

$P_{ij} = P(j|i) = X_{ij}/X_i$

GloVe wants to find word vectors W_i and context vectors W_j such that :

$$W_i^T W_j + \text{bias} = \log(X_{ij})$$



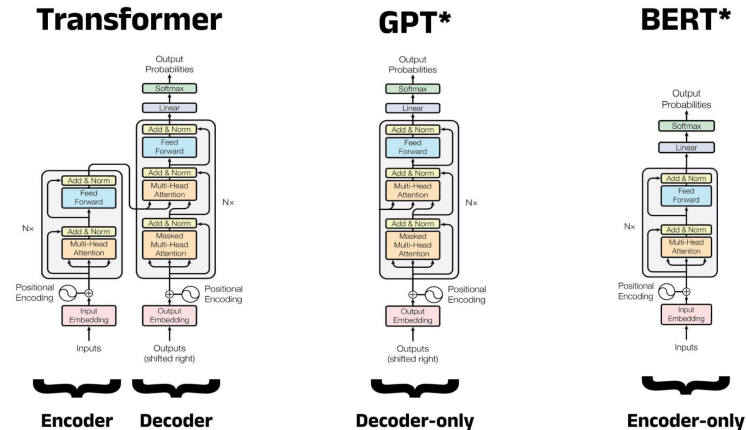
Embeddings

- **BERT(Bidirectional Encoder Representations from Transformers):**
 - **Neural network-based** produces contextual embeddings:
→ The same word gets different vectors depending on the sentence.
 - **Transformer based:** use only the encoder part of transformer architecture.
 - **Example:** for the word "light", BERT creates different embeddings in each sentence.
 - Please turn on the light in the kitchen (illumination device)
 - This suitcase is very light to carry (not heavy)
 - **Bidirectional:** It reads text in both directions (left-to-right and right-to-left) to better understand context.

Between 1886 and 1889, a German [REDACTED] named Heinrich Hertz conducted a series of brilliant experiments in a simple lab

Embeddings

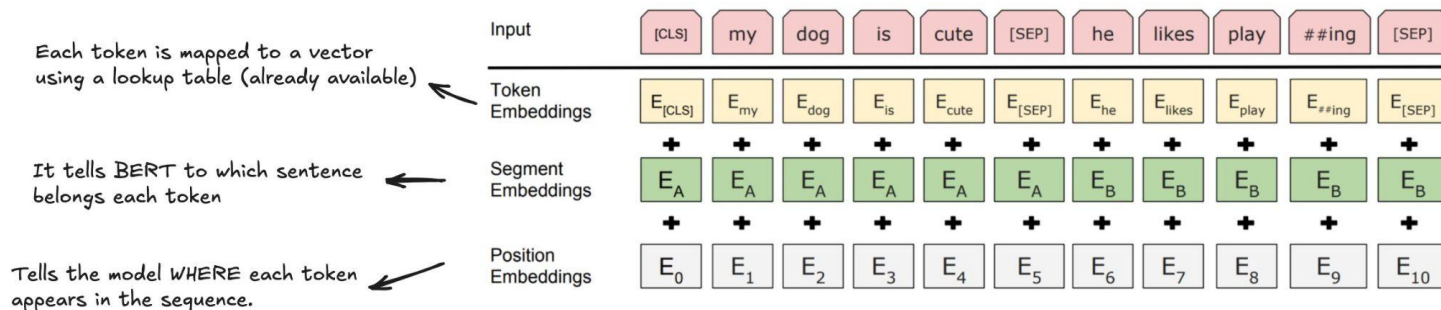
- BERT(Bidirectional Encoder Representations from Transformers):
 - Architecture



*Illustrative example, exact model architecture may vary slightly

Embeddings

- **BERT(Bidirectional Encoder Representations from Transformers):**
 - **BERT inputs:**
 - Sum of 3 vectors for each token => positional encoding, segment embedding, and token embedding





LAB3-2

- **Materials :**

- https://github.com/ababacaryoro/nlp-course/blob/main/lab3/lab3_2_word_embeddings.ipynb

- **Instructions :**

- Run the notebook (local with uv, Codespaces or Colab, see SETUP.md).
- Commit it with its outputs in your repository nlp-course-LASTNAME and push.
- Then notify the teacher by e-mail (repository link in your first e-mail only).



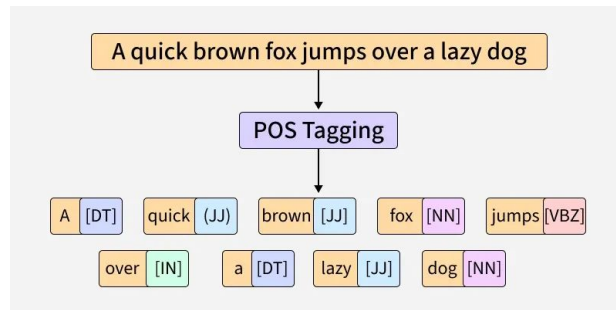
Part 4 : Core NLP tasks

Main NLP techniques

Use processed text for real-world applications

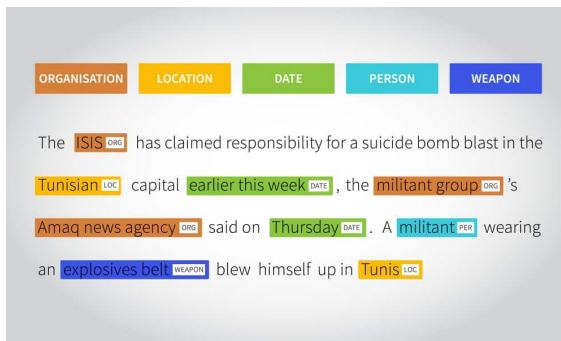
Part-Of-Speech (POS) tagging

- **POS tagging** assigns a **grammatical role** to each word.
 - Nouns, verbs, adjectives, prepositions etc.
- Underlying **techniques** to identify each class : using **dictionary** or a **predictive model** (markov chain for example).
- Example of classes :
 - **CC**: coordinating conjunction
 - **CD**: cardinal number
 - **DT**: determiner
 - **JJ**: adjective
 - **NN**: singular noun
 - **VBZ**: verb, 3rd person singular present
 - **IN**: preposition



Named Entity Recognition (NER)

- NER identifies real-world **entities** in text.
- **Examples** of entities: persons, organizations, locations, expressions of times, quantities, monetary values, percentages
- **Use cases** :
 - Find person names in a document (authors, complains of clients...)
 - Amount in a contract





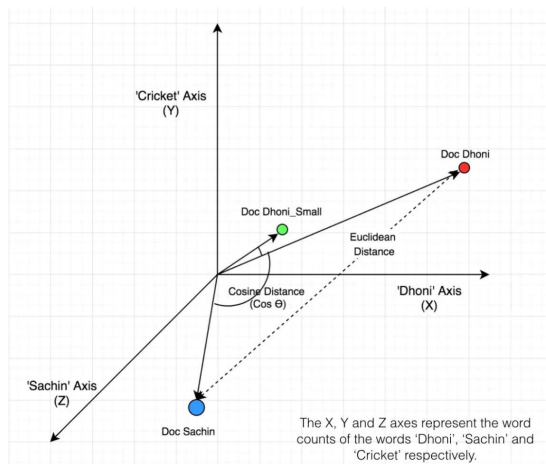
Similarity between words or docs

- **NLP** can measure how **similar** two **words**, **sentences**, or **documents** are.
- **Use cases:**
 - Search engines
 - Recommendation systems
 - Chatbots
- Most popular **methods:**
 - **Cosine** similarity (syntactic/semantic)
 - **Jaccard** Similarity (syntactic)
 - **Transformers based method** using distance metrics & embeddings (semantic)

Similarity between words or docs

- Cosine similarity

Vectorize words (embeddings or classical methods)



Compute cosine value

$$\text{Cos}\theta = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|} = \frac{\sum_1^n a_i b_i}{\sqrt{\sum_1^n a_i^2} \sqrt{\sum_1^n b_i^2}}$$

where, $\vec{a} \cdot \vec{b} = \sum_1^n a_i b_i = a_1 b_1 + a_2 b_2 + \dots + a_n b_n$ is the dot product of the two vectors.

Cosine Similarity Formula

Similarity between words or docs

- **Jaccard similarity:** how many words/characters the two texts have in common

Jaccard similarity score:

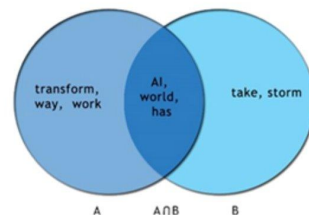
$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

where:

- A and B are sets of words
- $A \cap B$ = common words
- $A \cup B$ = total unique words

- **Example:**

- A = "AI has transformed the way the world works"
- B = "AI has taken the world by storm"



Jaccard similarity score: $\frac{3}{8} = 0.375 \rightarrow$ these texts are syntactically similar by 37.5%



LAB4-1

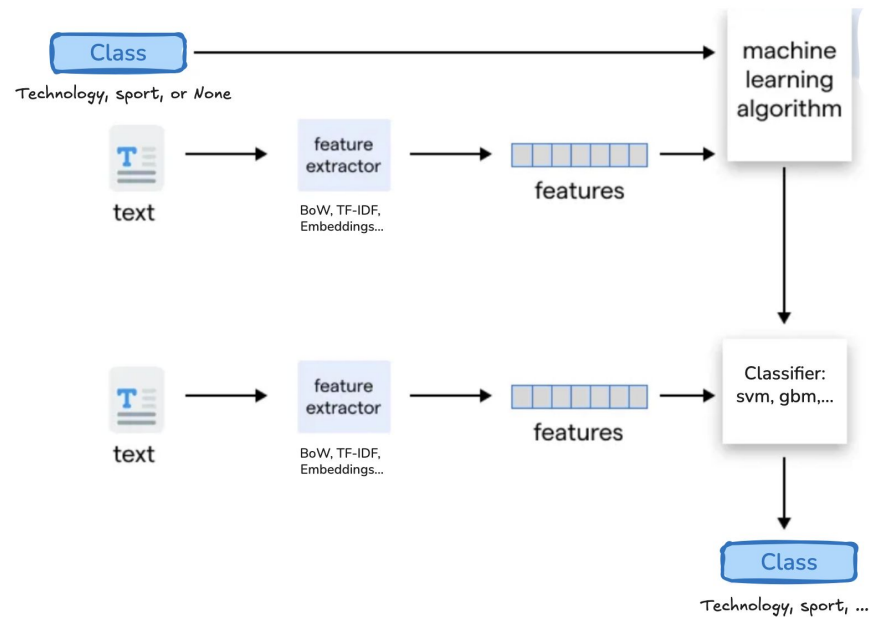
- **Materials :**
 - https://github.com/ababacaryoro/nlp-course/blob/main/lab4/lab4_1_core_nlp_tasks.ipynb
- **Instructions :**
 - Run the notebook (local with uv, Codespaces or Colab, see SETUP.md).
 - Commit it with its outputs in your repository nlp-course-LASTNAME and push.
 - Then notify the teacher by e-mail (repository link in your first e-mail only).



Document classification

- **Document classification** is the task of **assigning** one or more **labels** to a text.
- **Examples:**
 - Spam vs Not Spam
 - Sports / Politics / Business
 - Customer complaint / Praise / Question
- **Document Classification pipeline:**
 - Clean the text
 - Convert text to numbers (BoW, TF-IDF, Embeddings)
 - Train a model → Logistic Regression, Naive Bayes, SVM, GBM, Neural network, ...
 - Predict labels for new documents

Document classification





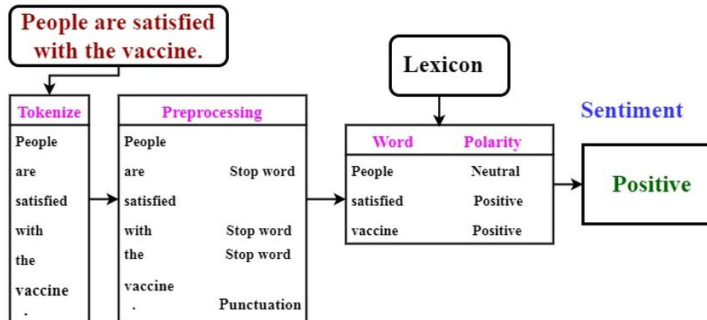
Sentiment analysis

- Sentiment analysis detects the **emotional tone** of a text.
- General sentiment **labels**:
 - Positive
 - Negative
 - Neutral
- **Approaches** in sentiment analysis:
 - **Rule/Lexicon based** methods
 - **Machine learning** methods
 - **Hybrid** methods

Sentiment analysis

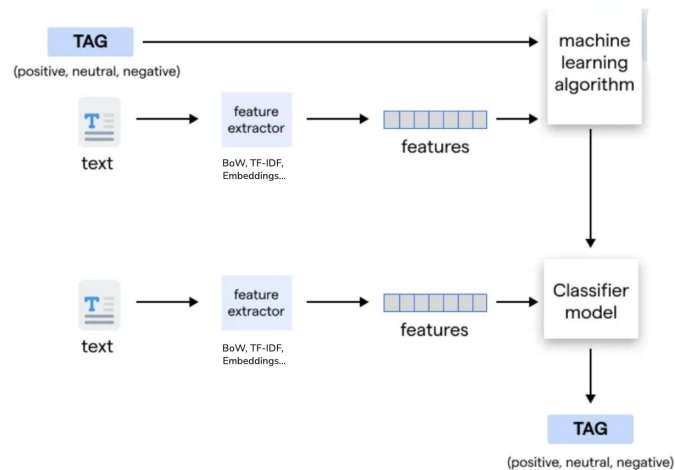
- Rule/Lexicon based methods:
 - Sentiment dictionaries →
 - Count the number of positive, negative and neutral words
 - Or get sentiment polarity score from words then aggregate

Input Text :



Sentiment analysis

- Machine learning methods:
 - Pipeline →
 - Clean text
 - Convert to vectors
 - Train or use a model
 - Predict sentiment



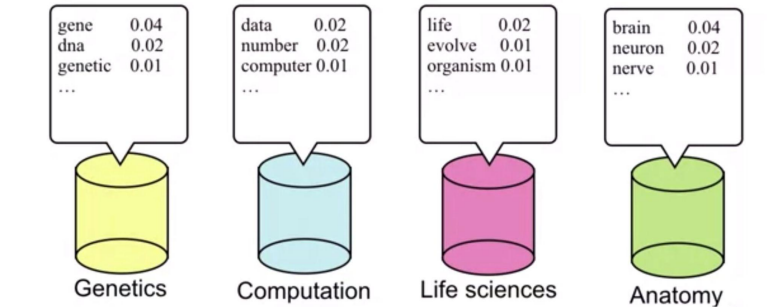


Topic modeling

- **Definition:** technique used to discover hidden thematic structure in documents
- **Motivation:**
 - We want to:
 - Discover main themes. Ex: news articles
 - Group similar documents. Ex: research papers
 - Explore trends over time. Ex: customer reviews
 - And large text corpora are hard to read manually
- **A topic?** → probability distribution over words
 - Example:
 - Topic 1: {economy, market, inflation, price}
 - Topic 2: {doctor, patient, hospital, treatment}

Topic modeling

- A topic? → probability distribution over words



- Main approaches for topic modeling:
 - Latent Dirichlet Allocation (LDA)
 - Non-negative Matrix Factorization (NMF)
 - Probabilistic Latent Semantic Analysis (pLSA)

Topic modeling

- LDA (Latent Dirichlet Allocation)

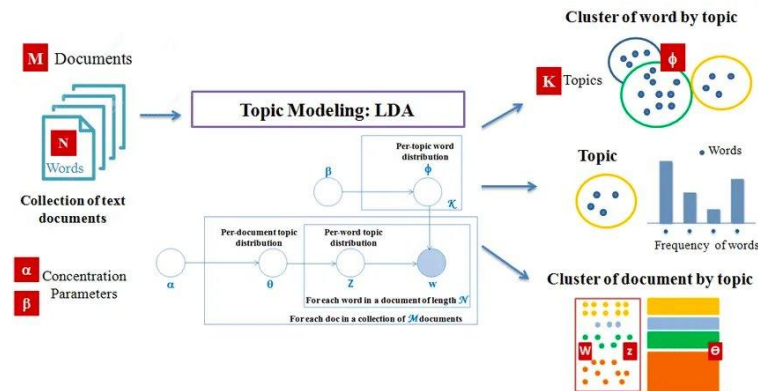
- A Probabilistic generative model

- Assumptions:

- Each document has a topic distribution
 - Ex: 70% Sport + 20% Finance + 10% Politics
- Each topic has a word distribution
 - Ex: "ball" (15%), "match" (12%), "team" (10%)...
- Words are generated by sampling topics

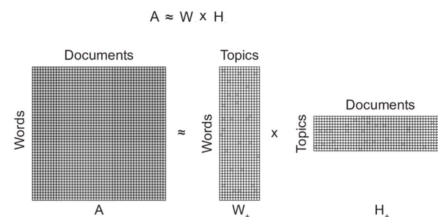
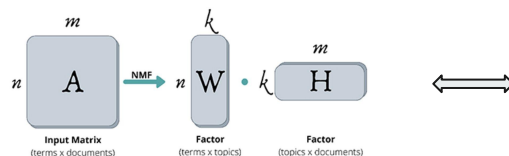
- Generative process:

- LDA does the opposite: based on the observed documents, it infers the topics and their mixtures
- Probabilistic models are used to model the distributions.



Topic modeling

- NMF (Non-negative Matrix Factorization)
 - A linear algebra-based method
 - Goal → decompose document-term matrix **A** into:
 - **W** (document-topic)
 - **H** (topic-word)



- Interpretation:
 - Topics are combinations of words
 - Documents are combinations of topics



LAB4-2

- **Materials :**

- https://github.com/ababacaryoro/nlp-course/blob/main/lab4/lab4_2_classification_sentiment_topics.ipynb

- **Instructions :**

- Run the notebook (local with uv, Codespaces or Colab, see SETUP.md).
- Commit it with its outputs in your repository nlp-course-LASTNAME and push.
- Then notify the teacher by e-mail (repository link in your first e-mail only).



Part 5 : Deep learning & LLMs for NLP

Neural networks usage

Advanced models for NLP

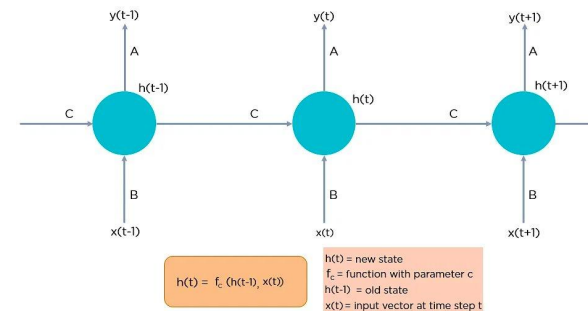
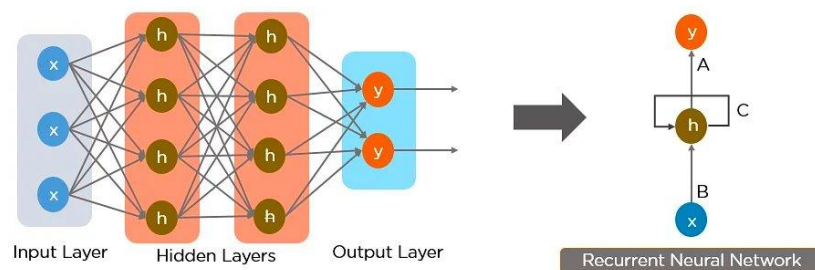


Neural network architectures for NLP

- **Why deep learning for NLP?**
 - Neural networks learn language patterns directly from data
 - Suitable for dense word representations
- **When?**
 - Once we have created Embeddings
 - When we do more complex tasks & need more specialization (sequence modeling): language generation, language translation
- **How?**
 - RNN (Recurrent Neural Networks)
 - LSTM (Long Short-Term Memory)
 - GRU (Gated Recurrent Units)
 - Transformers
 - ...

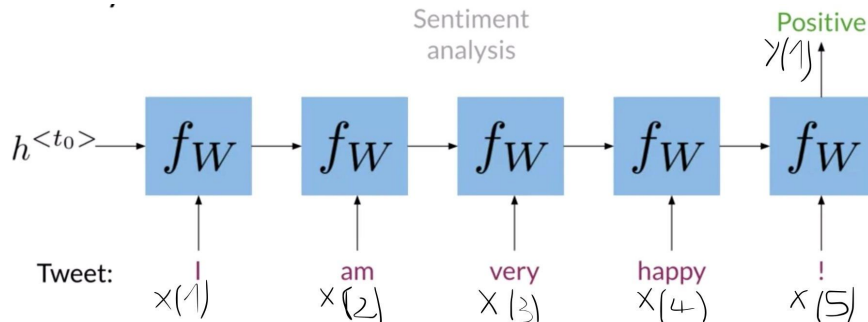
Neural network architectures for NLP

- RNN (Recurrent Neural Networks)
 - A neural network architecture that:
 - Maintains a hidden state which is updated at each time step by combining the previous hidden state with the current input
 - Enables the network to capture temporal/sequence patterns and dependencies.



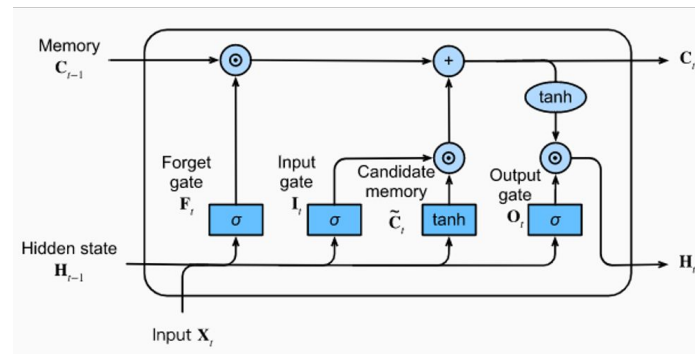
Neural network architectures for NLP

- RNN (Recurrent Neural Networks)
 - Use cases:
 - Language modeling
 - Sequence classification



Neural network architectures for NLP

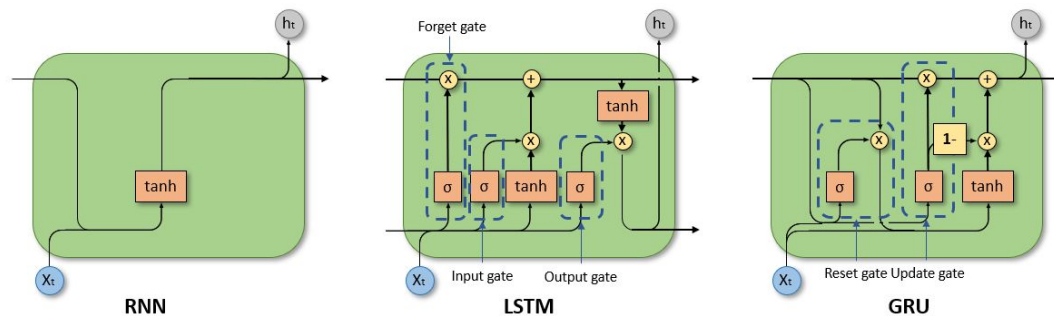
- LSTM (Long Short-Term Memory)
 - Basic RNN struggles to handle long-term dependencies
 - LSTM → introduces gates to control information flow:
 - **Forget gate:** what to remove from memory
 - **Input gate:** what new information to add
 - **Output gate:** what to output from memory



Architecture of a LSTM Unit

Neural network architectures for NLP

- GRU (Gated Recurrent Units)
 - Simplified alternative to LSTM
 - Fewer gates:
 - Reset gate: controls how much past information to forget
 - Update gate: controls how much of the previous hidden state to keep.
 - Faster training & comparable performance in many tasks



Neural network architectures for NLP

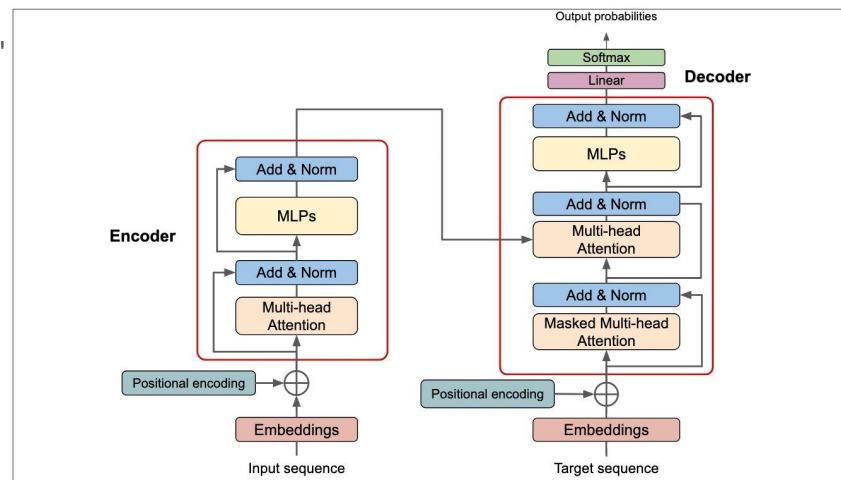
- **Transformers:**

- **Neural network architecture** that introduces the notion of “attention”:

- Not all **words** are equally **important**
- Model **learns** where to “pay attention”

- **Core components:**

- Token embeddings
- Positional encoding
- Multi-head self-attention
- Feed-forward layers

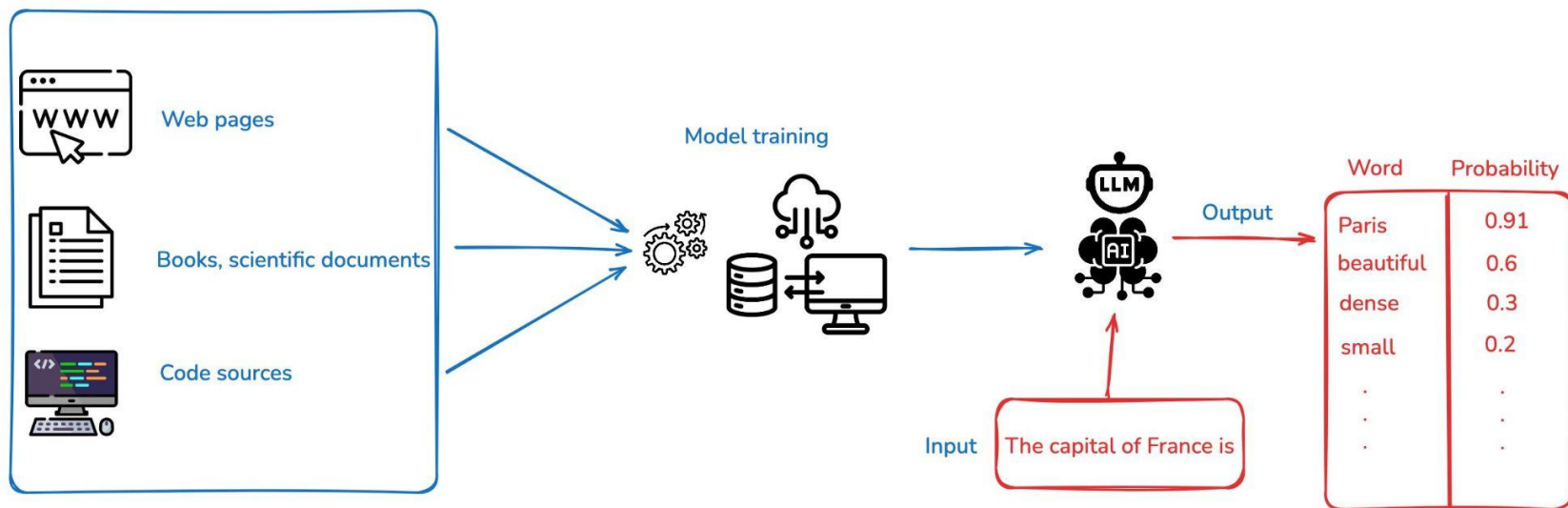




Large Language Models (LLMs)

- **Definition:** neural networks trained on massive text corpora to predict the next word in a sequence
- **Paradigm Shift:**
 - NLP → Task-specific models
 - LLMs → Universal language understanding
- **Key Characteristics:**
 - **Large:** Billions of parameters (GPT-3: 175B, GPT-4: estimations ~1,8T)
 - **Pre-trained:** Trained once on general text
 - **Transferable:** Fine-tuned for specific tasks
 - **Generative:** Can produce coherent text

Large Language Models (LLMs)



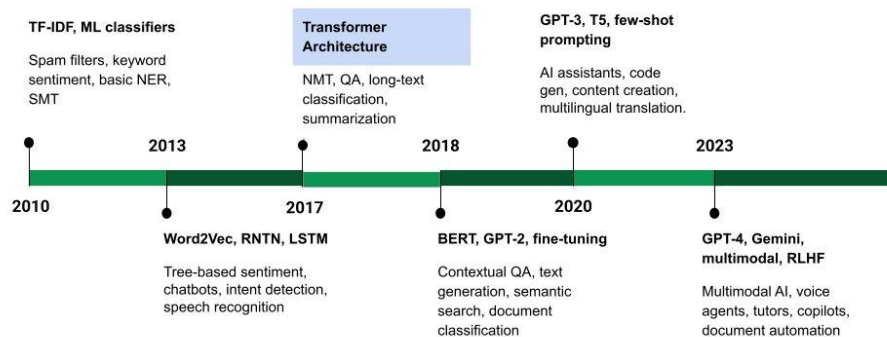


Large Language Models (LLMs)

- How it works :
 - **Training Data :**
 - LLMs are trained on trillions of words from books, articles, code, and the internet.
 - **Architecture :**
 - They use a Transformer architecture, which relies on self-attention to understand relationships between words and contexts across long passages.
 - **Learning Objective :**
 - They are trained to predict the next token (word or subword) given a context — this is called language modeling.
 - **Emergent Abilities :**
 - With enough parameters (billions) and data, LLMs start showing reasoning, summarization, translation, coding, and problem-solving skills.

Large Language Models (LLMs)

- Evolution of LLMs :

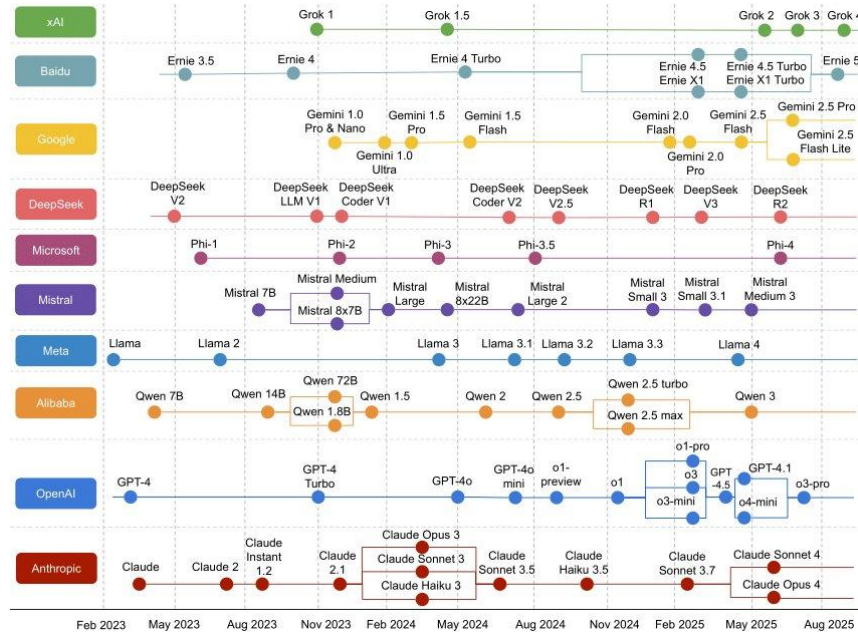


Post 2024:

Raisonnement models (o1/o3, DeepSeek-R1), Claude 4/5, GPT-5, Gemini 2.5/3, Mistral Medium 3, agents / tool use / MCP ...

Large Language Models (LLMs)

- Several players involved :





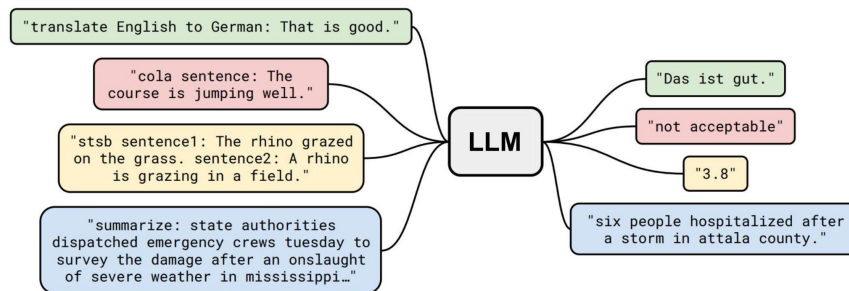
Large Language Models (LLMs)

- **Personalize LLMs :**
 - Prompt Engineering
 - Make clear, specific, and instructive prompts.
 - RAG – Retrieval-Augmented Generation
 - Combine the LLM with a search component that fetches relevant documents at query time.
 - Fine-Tuning or Instruction Tuning
 - Train the model further on your domain-specific QA data.
 - Agents / Tool calling :
 - Create programs that can reason, act, all around an LLM

Large Language Models (LLMs)

- **Prompt Engineering:**

- **Prompt:** input text or instruction given to a Large Language Model to guide its generation of output
- **Importance of prompts:** the quality and structure of the prompt directly impacts the model's output. Well-crafted prompts lead to accurate, relevant responses
- **Prompt Engineering :** the practice of designing effective prompts





Large Language Models (LLMs)

- **Prompt components:**
 - **Persona / tone:** e.g., “You are a cybersecurity assistant...”
 - **Instruction / task definition:** explicit statement of what the model should do.
 - **Context / background:** data, facts, or documents the model should use.
 - **Demonstrations:** examples of input → desired output.
 - **Constraints / format:** length limits, JSON schema, bullet points, forbidden content.
 - **Stop sequences / output markers:** to delimit the model response.

Prompt techniques : <https://www.promptingguide.ai/techniques>

Large Language Models (LLMs)

- Prompt components:

You are a professional email writer. Draft a polite rejection email to a job candidate named Sarah Chen who applied for a Marketing Manager position. Keep it under 100 words, maintain a respectful tone, and encourage future applications. Use this format:

Subject: [your subject line]
Body: [your email text]

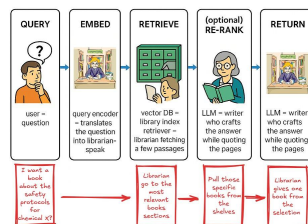


PERSONA:
"professional email writer"
TASK:
"Draft a polite rejection email"
CONTEXT:
Candidate name and position
CONSTRAINTS:
<100 words, respectful tone, encourage reapplication
OUTPUT FORMAT:
Specified structure (Subject/Body)

Large Language Models (LLMs)

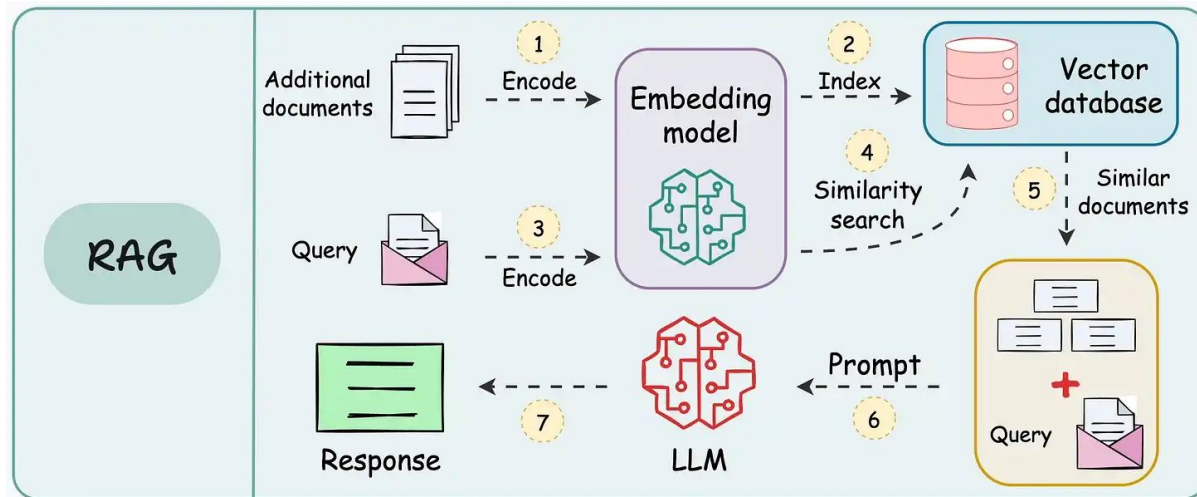
- **RAG – Retrieval-Augmented Generation:**
 - Objective : add new context information to the prompt for the LLM
 - Process : Index → Retrieve → Augment → Generate
 - Index: Index your documents (convert to embeddings, store in vector DB)
 - Retrieve: At query time, retrieve top relevant docs
 - Augment: Augment the prompt with retrieved context (optionally add citations)
 - Generate: LLM generates an answer grounded on augmented context

RAG Architecture — The Simple Pipeline (with an Analogy)



Large Language Models (LLMs)

- RAG – Retrieval-Augmented Generation:



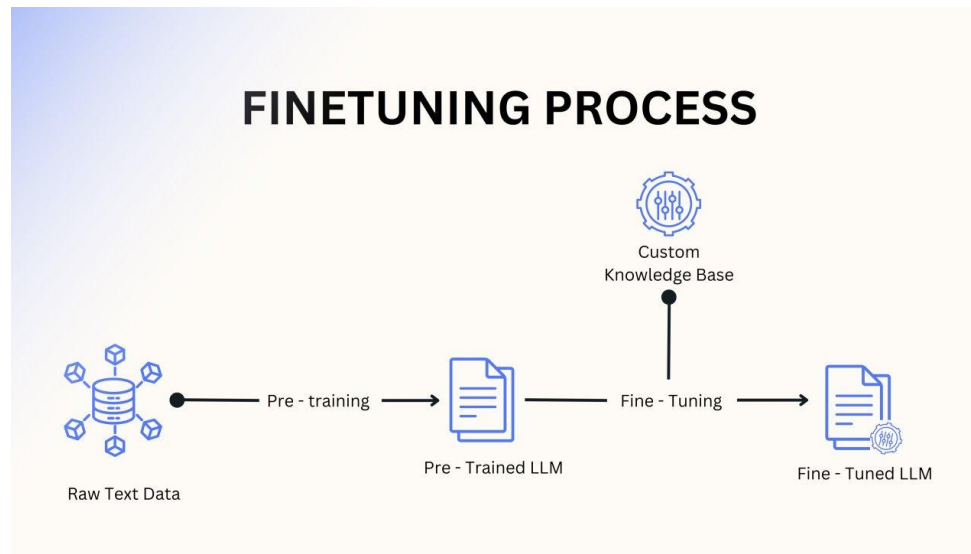


Large Language Models (LLMs)

- **Fine-Tuning (FT):**
 - **Definition:**
 - Adjusting a model's parameters (or adding small task-specific parameters) using labeled examples to change model behavior : LoRA / PEFT
 - **Objectives:**
 - Improve accuracy on target tasks, align tone/style/constraints, reduce hallucinations, embed domain knowledge.
 - **Trade-offs:**
 - Full FT = best task fit but high compute, storage, and versioning cost; requires data, compute, and safety review.

Large Language Models (LLMs)

- Fine-Tuning (FT):





LAB5

- **Materials :**

- https://github.com/ababacaryoro/nlp-course/blob/main/lab5/lab5_deep_learning_llms.ipynb

- **Instructions :**

- Run the notebook (local with uv, Codespaces or Colab, see SETUP.md).
- Commit it with its outputs in your repository nlp-course-LASTNAME and push.
- Then notify the teacher by e-mail (repository link in your first e-mail only).



Part 6 : Capstone project

NLP & LLMs

Real-world app



Capstone project

- **Description:**
 - **Objective:** Build a working application on real data with the techniques of the course, and present it.
 - **Organisation:** Teams of 2-3 or you can work alone
 - **Deliverables:** a public GitHub repository, a running web app (Gradio or Streamlit, local or on Hugging Face Spaces), a 10-minute presentation with a live demo.
 - **Instructions:**
 - <https://github.com/ababacaryoro/nlp-course/blob/main/capstone/README.md>
 - **Rules:**
 - Real data only;
 - At least one evaluation with a metric;
 - AI assistants allowed, but every member must explain every part. You should defend your choices